

Ensemble Learning

Bagging · Boosting · Random Forest · XGBoost

From Fundamentals to Mathematics — with Step-by-Step Diagrams

Decision Trees

Information Gain

Bias-Variance

Bagging

Boosting

XGBoost

Table of Contents

01 Prerequisites

Entropy · Information Gain · Decision Trees · Bias · Variance · Overfitting

02 Bagging

Bootstrap Aggregation — how averaging kills variance step by step

03 Random Forest

Bagging + Feature Randomness = Decorrelated Trees

04 Boosting

Sequential Error Correction — AdaBoost & Gradient Boosting

05 XGBoost

Regularised Gradient Boosting — the Competition Champion

06 Comparison

Side-by-side summary · When to use which

Prerequisites — Entropy: Measuring Disorder

CONCEPT

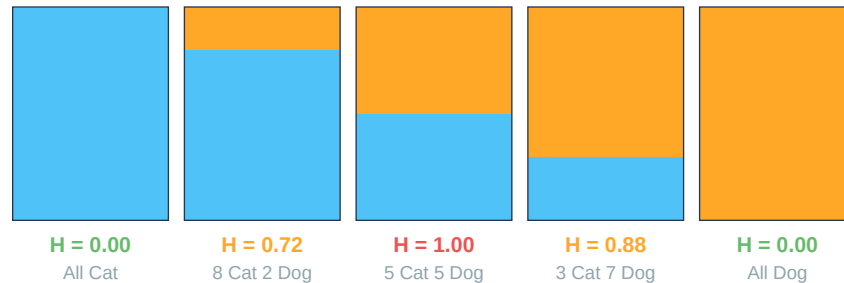
Entropy = How mixed / impure a set is

- Pure set (all same class): Entropy = 0
- Perfectly mixed (50/50): Entropy = 1
- More classes, more mixed = higher entropy

$$H(S) = - \sum p_i * \log_2(p_i)$$

p_i = fraction of class i in set S

VISUALISED



Prerequisites — Information Gain: Choosing the Best Split

CORE IDEA

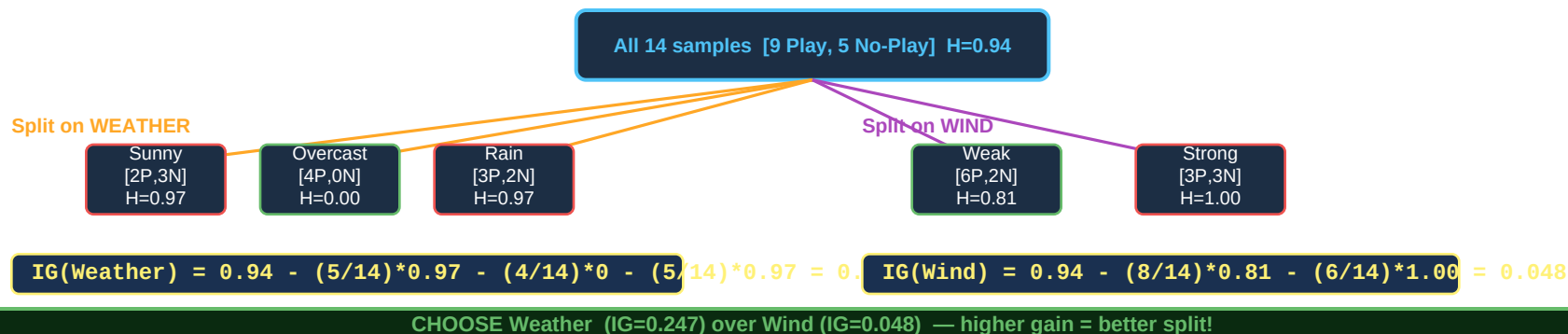
Information Gain = How much does a split REDUCE entropy?

- Before split: measure entropy of the whole set $H(S)$
- After split on feature A: measure weighted entropy of each child node
- $IG = H(S) - \text{weighted_avg}(H(\text{children}))$
- Pick the split with the HIGHEST Information Gain at each node

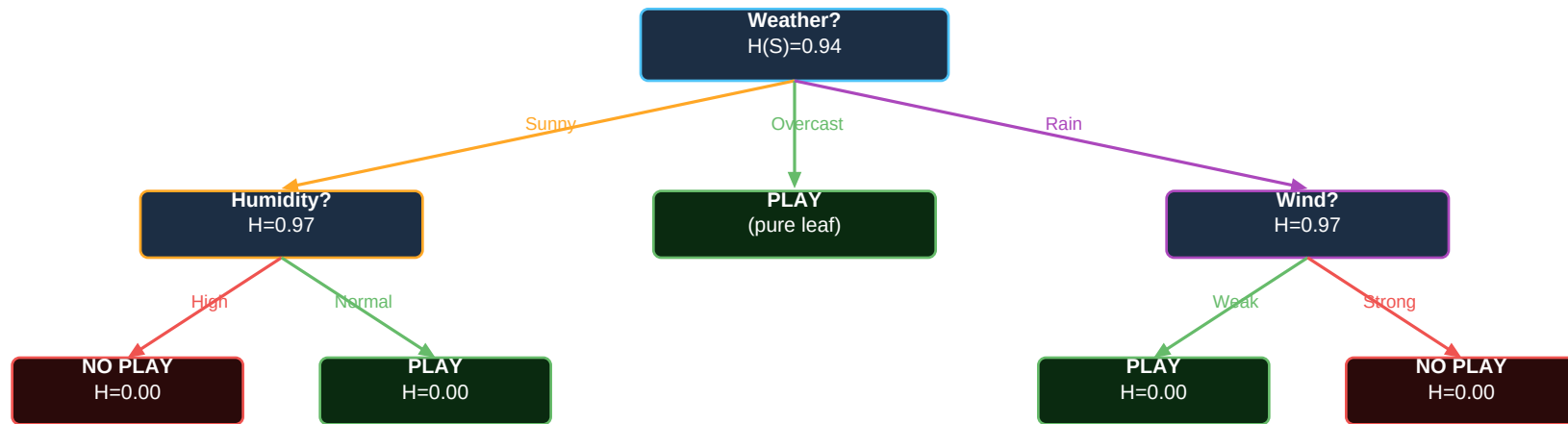
$$IG(S, A) = H(S) - \sum_v (|S_v|/|S|) * H(S_v)$$

S_v = subset of S where feature A has value v

WORKED EXAMPLE — SHOULD WE SPLIT ON WEATHER OR WIND?



Prerequisites — Building a Decision Tree with Information Gain



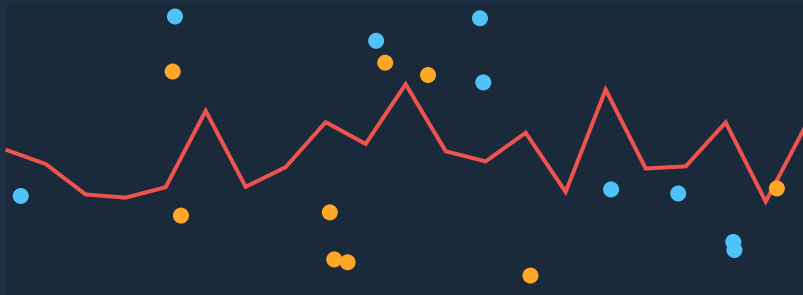
Algorithm: At each node → compute IG for ALL features → pick highest IG → split → recurse until pure or max_depth reached

Other metrics exist: Gini Impurity = $1 - \sum(p_i^2)$, Gain Ratio, Chi-Square. Information Gain is the most interpretable.

Prerequisites — The Problem: Decision Trees Overfit

PROBLEM SETUP

Deep Tree → OVERFITTING



Boundary memorises NOISE
Fails on new data (HIGH VARIANCE)

Train accuracy: ~100% Test accuracy: ~65%

Bias: LOW Variance: HIGH

Shallow Tree → UNDERFITTING



Boundary too simple — misses structure
Poor on train AND test (HIGH BIAS)

Train accuracy: ~72% Test accuracy: ~70%

Bias: HIGH Variance: LOW

Prerequisites — The Bias-Variance Tradeoff

THE DECOMPOSITION

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- Bias²: error from wrong assumptions (too simple model)
- Variance: error from sensitivity to training data fluctuations
- Irreducible noise: cannot be eliminated — it's in the data
- Bias and Variance TRADE OFF — reducing one tends to increase the other
- Ensemble methods BREAK this tradeoff by attacking each separately

TARGET ANALOGY



High Bias
Low Variance



Low Bias
High Variance



Low Bias
Low Variance
(GOAL)



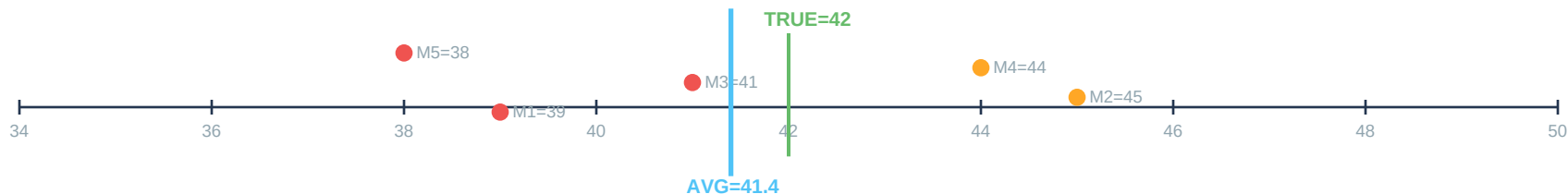
High Bias
High Variance
(Worst)

Prerequisites — The Big Idea: Combine Many Weak Models

Wisdom of Crowds: Average of many imperfect predictions beats any single prediction

Each model has its own error — but errors are RANDOM and cancel each other out when averaged

ILLUSTRATION — 5 MODELS EACH PREDICT A VALUE



Each model's ERROR is different (some too high, some too low).

When we AVERAGE, the errors partially cancel → the average is CLOSER to the truth than any individual model.

BAGGING

Parallel training. Each model sees different data. Average/vote. Best for HIGH VARIANCE.

Fixes: Overfitting

BOOSTING

Sequential training. Each model corrects the previous one's mistakes. Best for HIGH BIAS.

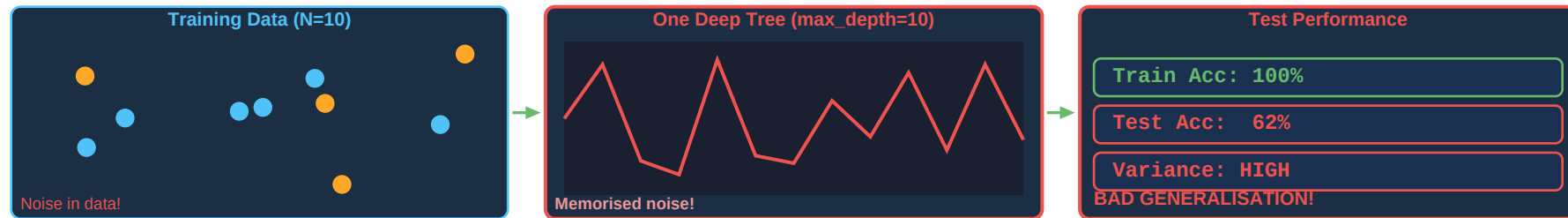
Fixes: Underfitting

Bagging 02

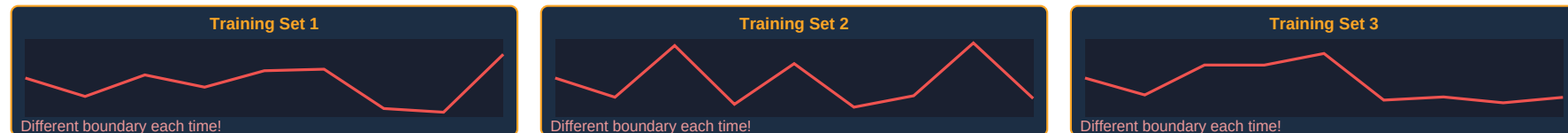
Bootstrap Aggregation — Killing Variance One Tree at a Time

Bagging — Step 1: The Problem (One Deep Tree Overfits)

PROBLEM DIAGRAM

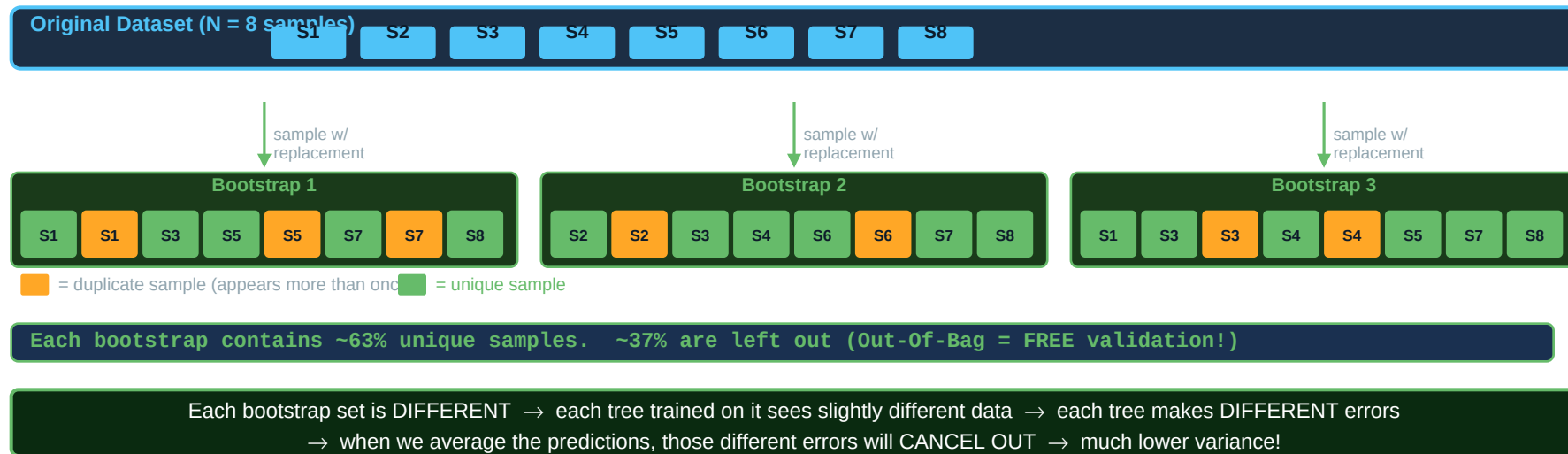


Key observation: train same deep tree on slightly different data → COMPLETELY different boundaries! The model is highly unstable.



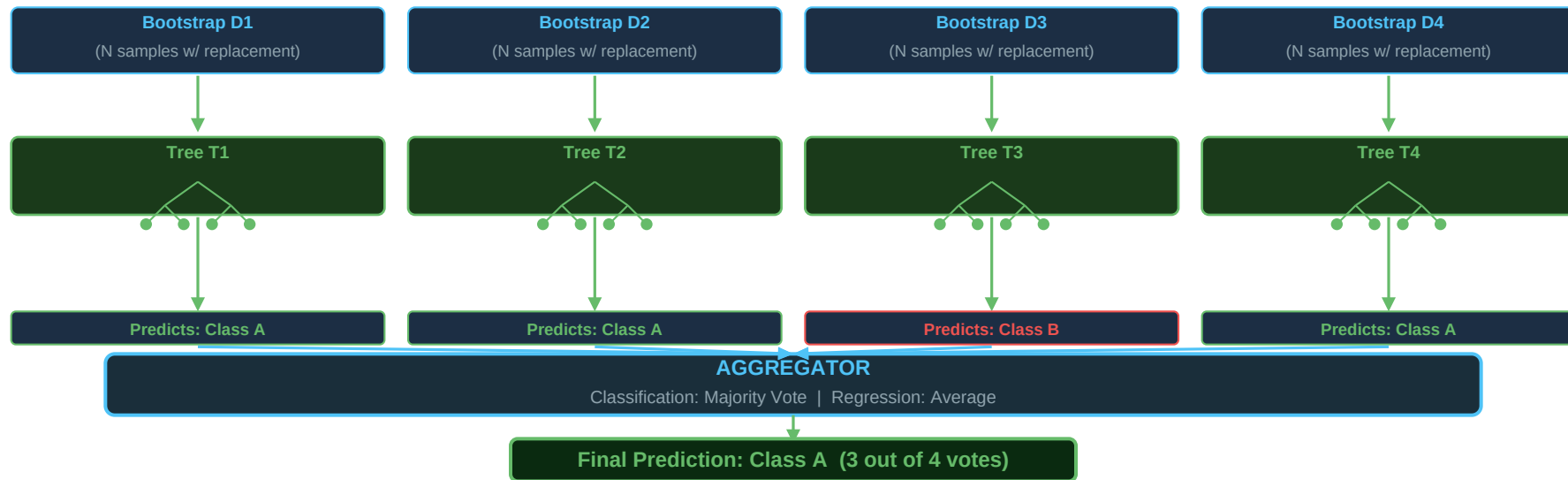
Bagging — Step 2: Bootstrap Sampling (The Fix Begins)

SOLUTION STEP 2A — CREATE B DIFFERENT TRAINING SETS FROM ONE DATASET



Bagging — Step 3 & 4: Train Trees + Aggregate Predictions

SOLUTION STEP 2B — TRAIN ONE TREE PER BOOTSTRAP, THEN COMBINE



$f_{\text{bag}}(x) = \text{majority_vote}\{T_1(x), T_2(x), \dots, T_B(x)\}$ or $(1/B) \cdot \sum T_b(x)$ for regression

Bagging — Why Variance DROPS: The Math Explained Visually

PROOF — WHY DOES AVERAGING REDUCE VARIANCE?

Each tree's prediction = truth + its own random error:

$$T_b(x) = f^*(x) + \text{epsilon}_b \quad \text{where } \text{epsilon}_b \text{ is random error with mean}=0$$

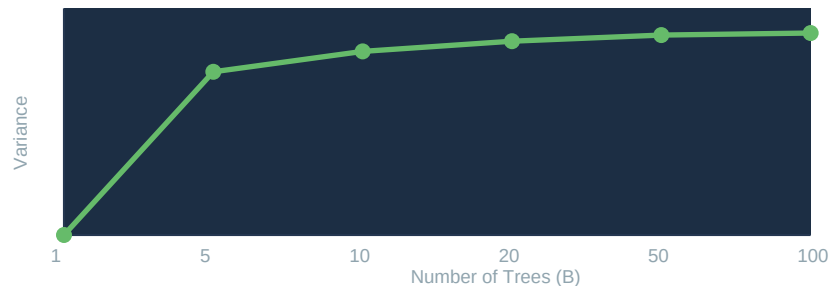
When we AVERAGE B trees:

$$f_{\text{bag}}(x) = (1/B) * \text{SUM } T_b(x) = f^*(x) + (1/B) * \text{SUM } \text{epsilon}_b$$

$$\text{Var}(f_{\text{bag}}) = (1/B^2) * \text{SUM } \text{Var}(\text{epsilon}_b) = \sigma^2 / B \quad (\text{if trees independent})$$

Conclusion: More trees → Variance goes down by factor of B → LESS OVERFITTING!

VARIANCE REDUCTION AS B INCREASES



What about BIAS?

Bias stays the SAME.
Averaging unbiased predictions
gives an unbiased average.

Bagging does NOT fix underfitting.
Use Boosting for that!

Bagging solves OVERFITTING (High Variance). It does NOT reduce Bias.

Random Forest

Bagging + Feature Randomness = Decorrelated Trees

03

Random Forest — The Problem with Plain Bagging: Correlated Trees

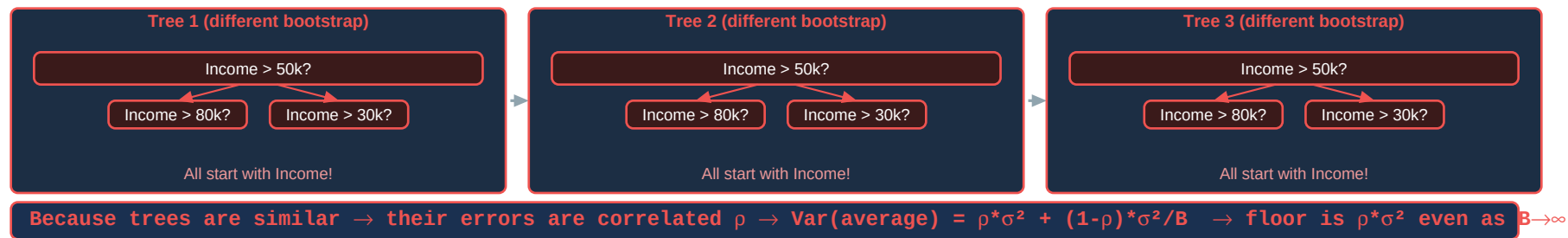
NEW PROBLEM — BAGGING TREES CAN BE CORRELATED!

Even with bootstrap sampling, all trees see the same pool of features.

If one feature is very strong (e.g. 'Income'), ALL trees will split on it first → similar trees → correlated errors!

Correlated errors do NOT cancel out when averaged. Variance reduction is limited!

ILLUSTRATION — ALL TREES CHOOSE THE SAME DOMINANT FEATURE



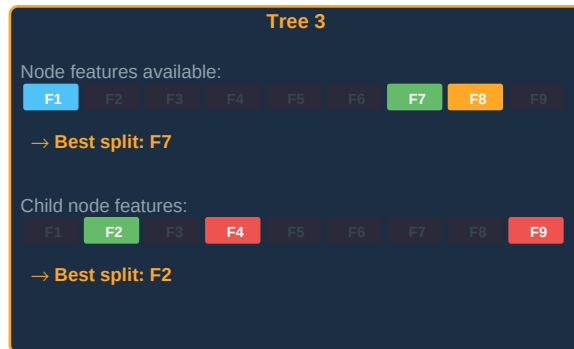
Solution: FORCE trees to be different by randomising which features they can use at each split!

Random Forest — The Fix: Random Feature Subsets at Every Split

SOLUTION — AT EACH SPLIT, ONLY CONSIDER M FEATURES CHOSEN AT RANDOM

$m = \sqrt{p}$ for classification $m = p/3$ for regression $p = \text{total number of features}$

Suppose we have 9 features total ($p=9$). At each node, randomly pick $m=3$ features. Build the split using only those 3.



Each tree chooses from a DIFFERENT random subset → different root splits → DECORRELATED trees!

Even if Income is the globally best feature, Tree 2 might not even see it → forced to find the second-best split!

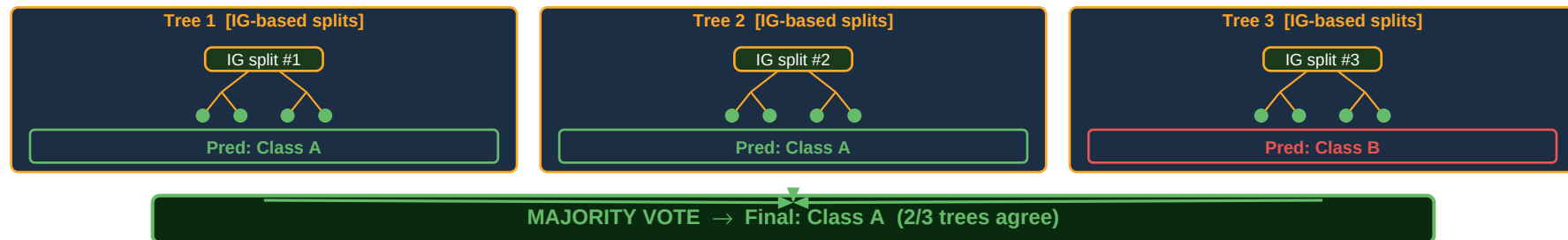
$\text{Var(RF)} = \rho \cdot \sigma^2 + (1-\rho) \cdot \sigma^2 / B$ Lower ρ = lower variance floor!

Random forests lower ρ by forcing diverse splits → variance drops FURTHER than plain Bagging!

Random Forest — Complete Algorithm Diagram



FULL ILLUSTRATION WITH 3 TREES



OOB Score: each sample predicted by trees that did NOT train on it → free cross-validation accuracy!

Boosting 04

Sequential Error Correction — Fighting Bias, Round by Round

Boosting — The Problem: Bagging Can't Fix High Bias

THE PROBLEM — SHALLOW TREES (STUMPS) UNDERFIT. AVERAGING THEM DOESN'T HELP.

A shallow 1-level tree (stump) can only learn one simple rule. It misses most patterns (HIGH BIAS).

If we bag 100 shallow stumps... we just average 100 simple rules → still a simple rule. BIAS IS NOT REDUCED.



Averaging all 3 stumps → still the same horizontal line → **STILL HIGH BIAS!**

We need a **DIFFERENT** approach: each new model must **CORRECT** the mistakes of the previous ones.

Key insight for Boosting: instead of training models independently, train them **SEQUENTIALLY** where each one focuses on what the last one got **WRONG**.

Boosting — AdaBoost: Reweighting Mistakes Step by Step

8 TRAINING SAMPLES — INITIAL WEIGHTS ALL EQUAL (1/8)

Round 1: Stump on Feature A



Round 2: Focus on wrong ones → Stump B



Round 3: Focus on remaining errors → Stump C



$$F(x) = \text{sign}(\alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x)) \quad \text{Higher } \alpha = \text{more trusted model}$$

AdaBoost — Full Mathematics

INITIALISATION & ALGORITHM

$w_i = 1/N$ for all $i=1..N$

1. Train weak learner h_t

on weighted samples

2. Compute weighted error

$\text{eps}_t = \text{SUM}(w_i * I(h_t \text{ wrong})) / \text{SUM}(w_i)$

3. Compute model weight

$\alpha_t = 0.5 * \ln((1-\text{eps}_t) / \text{eps}_t)$

4. Update weights

$w_i = w_i * \exp(-\alpha_t * y_i * h_t(x_i))$

5. Normalise weights

$\text{SUM}(w_i) = 1$ (divide all by sum)

Final: $F(x) = \text{sign}(\text{SUM } \alpha_t * h_t(x))$

ALPHA INSIGHT



- $\text{eps} < 0.5 \rightarrow \alpha > 0 \rightarrow$ model helps
- $\text{eps} = 0.5 \rightarrow \alpha = 0 \rightarrow$ model ignored (random)
- $\text{eps} \rightarrow 0 \rightarrow \alpha \rightarrow \infty \rightarrow$ near-perfect model gets huge weight
- $\text{eps} > 0.5 \rightarrow \alpha < 0 \rightarrow$ model's vote is FLIPPED

Boosting — Gradient Boosting: Fitting Residuals Step by Step

KEY IDEA — FIT EACH NEW TREE TO THE RESIDUALS (ERRORS) OF THE CURRENT MODEL



NUMERIC EXAMPLE (3 SAMPLES, REGRESSION)

Sample	True y	F0 (mean)	F0 residual	F1 pred	F1 residual
x=1	10	8.0	+2.0	9.8	+0.2
x=2	20	8.0	+12.0	19.6	+0.4
x=3	15	8.0	+7.0	14.8	+0.2

$F_m(x) = F_{\{m-1\}}(x) + \eta \cdot h_m(x)$ Residuals shrink each round → Bias reduces!

Boosting — Fixes Bias But Can Overfit: How to Control It

WHAT BOOSTING SOLVES & ITS RISKS

FIXES: High Bias (Underfitting)

- Sequential correction reduces bias each round
- Weak stumps combined → strong model
- Can model complex non-linear patterns
- More rounds (M) → lower training error

RISK: Overfitting if not controlled

- Too many rounds M → memorises training data
- Learning rate too high → overly aggressive
- Deep trees → each step too complex

How to prevent overfitting:

Shrinkage: Small learning rate η (0.01–0.1)

Early stopping: Stop when val error increases

Subsampling: Use fraction of rows per round

Shallow trees: `max_depth = 3 or 4` only

XGBoost 05

extreme Gradient Boosting — Regularised, Fast, and Dominant

XGBoost — The Problem: Vanilla Gradient Boosting Overfits

THE PROBLEM — STANDARD GB HAS NO REGULARISATION AND IS SLOW

Vanilla Gradient Boosting (Friedman 2001) works well, but has key weaknesses:

No regularisation: Leaf weights can grow arbitrarily large → overfitting

Greedy splits only: Uses only first-order gradient → slow convergence

No missing value handling: Must pre-process missing data manually

Sequential (slow): Cannot parallelise tree building

Heuristic split finding: Exact greedy is $O(N \cdot p)$ per split — slow on large data

XGBoost (Chen & Guestrin, 2016) adds: L1+L2 regularisation + 2nd-order gradients + column subsampling + cache-aware computation

XGBoost — The Fix: Regularised Objective + 2nd Order Gradients

THE SOLUTION — NEW OBJECTIVE FUNCTION WITH BUILT-IN REGULARISATION

$$\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Loss Term: $\sum L(y_i, \hat{y}_i)$

How well does model fit?

MSE: $(y - \hat{y})^2 / 2$

Log Loss for classification

→ Want this to go DOWN

Regularisation: $\Omega(f)$

How complex is the tree?

$\Omega(f) = \gamma T + (\lambda/2) \sum w_j^2$

T=leaves, w_j =leaf weights

→ Penalises complexity → prevents overfit

2ND ORDER TAYLOR APPROXIMATION — SMARTER UPDATES

$$\text{Obj}_{\text{approx}} = \sum_i [g_i f(x_i) + (1/2) h_i f(x_i)^2] + \Omega(f)$$

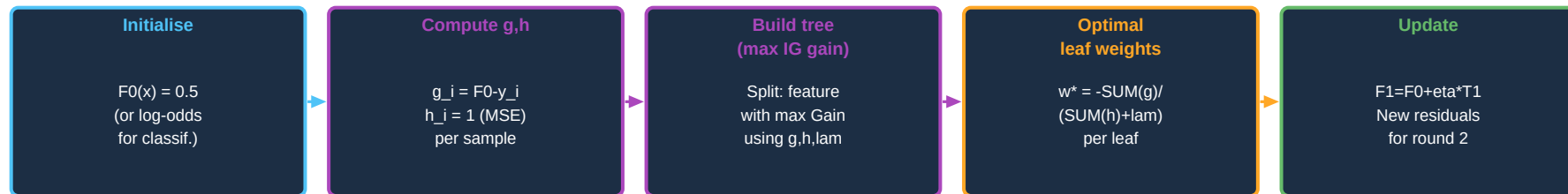
$$g_i = dL/d\hat{y} \quad (1\text{st deriv, direction}) \quad h_i = d^2L/d\hat{y}^2 \quad (2\text{nd deriv, curvature})$$

Closed-form optimal leaf weight & split gain:

$$w_j^* = -(\sum_j g_i) / (\sum_j h_i + \lambda) \quad \text{Gain} = (1/2) * [G_L^2 / (H_L + \lambda) + G_R^2 / (H_R + \lambda)] - (G_L + G_R)^2 / (H_L + H_R + \lambda)$$

XGBoost — Step-by-Step Pipeline (Numeric Example)

EXAMPLE: 4 SAMPLES, PREDICT HOUSE PRICE. $\eta=0.3$, $\lambda=1$, $\gamma=0$



NUMERICAL TRACE — 2 ROUNDS

x	True y	F0	g = F0-y	h=1	Leaf w*	F1=F0+0.3*w*	New residual
1	10	0.5	-9.5	1	-9.5/2=-4.75	0.5+0.3*-4.75=-0.92	10-(-0.92)=10.92
2	15	0.5	-14.5	1	-14.5/2=-7.25	0.5+0.3*-7.25=-1.68	15-(-1.68)=16.68
3	8	0.5	-7.5	1	-7.5/2=-3.75	0.5+0.3*-3.75=-0.63	8-(-0.63)=8.63
4	12	0.5	-11.5	1	-11.5/2=-5.75	0.5+0.3*-5.75=-1.22	12-(-1.22)=13.22

Note: w^* uses $\text{SUM}(g) / \text{SUM}(h) + \lambda$ PER LEAF — here all 4 are in one leaf for simplicity. γ penalises adding more leaves.

XGBoost — Hyperparameters & How They Fight Overfitting/Underfitting

<code>n_estimators</code>	M	Number of trees / boosting rounds	≈ 100	More = lower bias. Too many = overfit.
<code>learning_rate</code>	eta	Step size — shrinks each tree's contribution	≈ 0.1	Smaller = better generalisation, needs more trees
<code>max_depth</code>	d	Max depth of each tree	≈ 6	Deeper = more bias reduction, more overfit risk
<code>gamma</code>	γ	Min gain needed to make a split	≈ 0	Higher = fewer splits = simpler trees = less overfit
<code>reg_lambda</code>	λ	L2 ridge on leaf weights	≈ 1	Higher = smaller leaf weights = less overfit
<code>reg_alpha</code>	α	L1 lasso on leaf weights	≈ 0	Sparsifies weights — useful with many features
<code>subsample</code>	ss	Fraction of rows sampled per tree (Stochastic GB)	≈ 1	< 1.0 adds randomness → reduces overfit
<code>colsample_bytree</code>	cst	Fraction of features used per tree (like RF)	≈ 1	< 1.0 decorrelates trees → reduces overfit
<code>min_child_weight</code>	mcw	Min hessian sum in leaf (complexity control)	≈ 1	Higher = simpler model = less overfit
<code>early_stopping</code>	es	Stop if val metric doesn't improve for N rounds	≈ 10	Best practice: always use with a validation set

Comparison

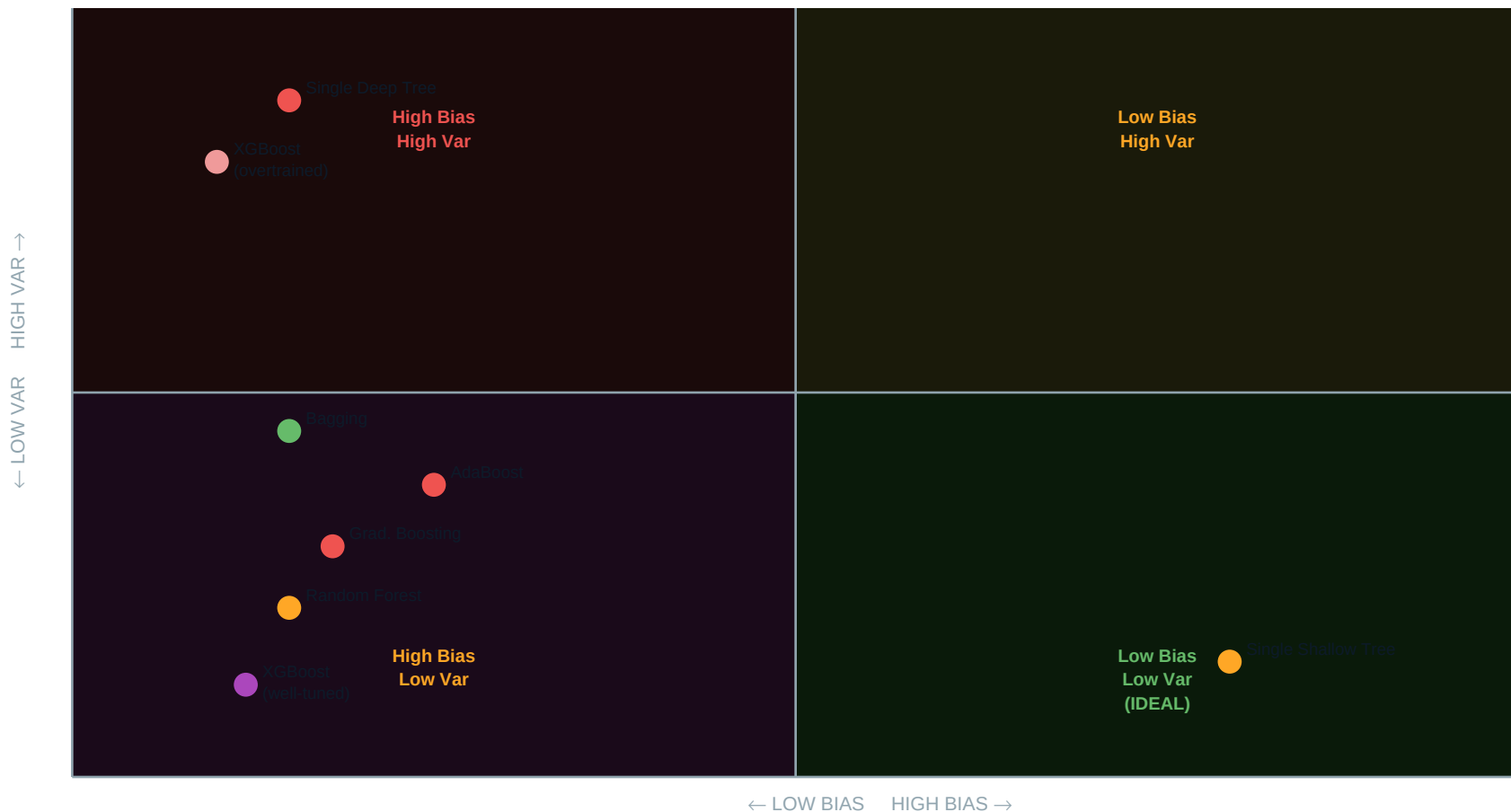
Side-by-side guide — know when to use what

06

Full Comparison — All Methods Side by Side

Property	Bagging	Random Forest	AdaBoost	GradBoost	XGBoost
Training	Parallel	Parallel	Sequential	Sequential	Sequential
Base learner	Any	Dec. Trees	Stump/tree	Shallow tree	Shallow tree
Fixes	Variance	Variance	Bias	Bias+Var	Bias+Var
Aggregation	Vote/Mean	Vote/Mean	Wtd. vote	Additive	Additive
Regularisation	None	Via depth	Via alpha	Shrinkage	L1+L2+gamma
Missing data	Manual	Manual	Manual	Manual	Built-in
Feature import	No	YES	Partial	YES	YES
Overfit risk	Low	Very Low	Can overfit	Medium	Controllable
Speed	Fast	Fast	Medium	Slow	Very Fast
Best for	High-var mdl's	Tabular data	Weak lnr's	Complex data	Most tasks

Visual Summary — Where Each Method Sits on Bias-Variance Map



Decision Guide — When to Use Which Algorithm

Model is OVERFITTING (high variance)

→ Bagging / Random Forest

Model is UNDERFITTING (high bias)

→ Boosting (AdaBoost/XGBoost)

Fast training needed, good baseline

→ Random Forest

Winning competitions on tabular data

→ XGBoost / LightGBM

Dataset has missing values, no preprocessing

→ XGBoost (handles natively)

Need feature importance ranking

→ Random Forest or XGBoost

Very high dimensional sparse features

→ XGBoost with `colsample < 1`

Interpretability required

→ Single Decision Tree (not ensemble)

Small dataset, noisy labels

→ Random Forest (OOB validation)

Need uncertainty estimates

→ Bagging (prediction variance across trees)

Key Takeaways — Everything in One Place

Information Gain: $H(S) - \sum(|S_v|/|S| * H(S_v))$ is the primary split metric in decision trees. Other metrics (Gini, Chi-sq) also exist.

Bagging: PARALLEL. Bootstrap + average. Reduces VARIANCE (fixes overfitting). Bias unchanged.

Random Forest: Bagging + random feature subsets at each split. Decorrelates trees → lower variance floor.

Boosting: SEQUENTIAL. Each model corrects previous errors. Reduces BIAS (fixes underfitting). Risk of overfit.

XGBoost: Regularised GB + 2nd order gradients + column sampling. Controls both bias AND variance. Fastest.

Rule of thumb: Overfit? → Add trees to Bagging/RF or regularise. Underfit? → Boost longer or deeper.